STUDENT LABORATORY MANUAL

Module 5: Introduction to Computer Graphics and Visual Computing

20 Guided Examples • 20 Practice Exercises • 20 Outputs

Course: CS0045 – Computer Graphics and Visual Computing

Companion: Module 5 Student Guide

Toolchain: VS Code + MSYS2 UCRT64 GCC + GLEW + FreeGLUT

Student Name:	**Ebron, Nicole Anne V.**
Section:	**TN35**
Professor:	**Dr. Hadji J. Tejuco**

GitHub Repository: https://github.com/nicoleanneebron-art/Ebron-Nicole-Anne-V.

How to Use This Manual

This manual has three parts:

- Part A — 20 worked Examples (Ex01–Ex20). Each is a complete, compiled, ready-to-run program demonstrating one concept at a time: the 5-step VBO conversion, buffer usage hints, indexed drawing with an element buffer, interleaved buffers, glMapBuffer editing, timer-driven animation, and a final capstone scene. Type or copy each one into your SOURCE CODE\M5 folder, compile it (Ctrl+Shift+B), and run it (F5) before moving to the next.
- Part B — 20 Practice Exercises (Q01–Q20), split into Easy (Q01–Q07), Medium (Q08–Q14), and Hard (Q15–Q20). Each exercise gives you an Objective, a Task question describing exactly what your program must do, and Hints to point you in the right direction. Read the task and hints, then write the program YOURSELF before checking Part C.
- Part C — the full Answer Key for every exercise in Part B, with complete, compiled, working code.

Setup reminder

This module requires GLEW in addition to FreeGLUT. On Windows/MSYS2 UCRT64, install it with: pacman -S mingw-w64-ucrt-x86_64-glew. Include <GL/glew.h> BEFORE <GL/glut.h>, and call glewInit() after glutCreateWindow()/glutDisplayFunc() but before glutMainLoop(). All programs in this manual use the same cross-platform header block shown below.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include  <iostream>
using namespace std;
```

Every program in this manual has been compiled and verified with g++ against GLEW and FreeGLUT.

Part A — Guided Examples (Ex01–Ex20)

Example 01 — Immediate Mode Baseline (Before Conversion)

Concept: A plain glBegin()/glEnd() triangle — the starting point for Example 02's conversion to a Vertex Buffer Object.

Expected output: A solid green triangle, drawn the immediate-mode way.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 01 - Immediate Mode Baseline (Before Conversion)
 * ---------------------------------------------------------
 * Concept: A plain glBegin()/glEnd() triangle. This is the starting
 * point Example 02 converts into a Vertex Buffer Object (VBO), so you
 * can compare both approaches side by side.
 */

void triangle() {
    glBegin(GL_TRIANGLES);
        glVertex2f(0.0f, 0.75f);
        glVertex2f(-0.75f, 0.0f);
        glVertex2f(0.75f, 0.0f);
    glEnd();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor4f(.16f, .72f, .08f, 1.0f);
    triangle();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");
```

```
    glutDisplayFunc(display);


#ifndef __APPLE_
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 02 — The Same Triangle, as a Vertex Buffer Object

Concept: The 5-step VBO pattern: glGenBuffers -> glBindBuffer -> glBufferData -> glVertexPointer(with a byte OFFSET) -> glDrawArrays -> cleanup.

Expected output: The identical green triangle from Example 01, now stored in GPU (server) memory and rendered from a VBO.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 02 - The Same Triangle, as a Vertex Buffer Object (VBO)
 * ------------------------------------------------------------
 * Concept: The 5-step VBO pattern:
 *   1. Initialize the vertex data in an ordinary array.
 *   2. Generate a buffer ID and bind it: glGenBuffers, glBindBuffer,
 *      glBufferData - this UPLOADS the array to the GPU's own memory.
 *   3. glEnableClientState(GL_VERTEX_ARRAY).
 *   4. glVertexPointer(...) + glDrawArrays(...) - note the pointer
 *      argument is now an OFFSET (0), not a real pointer, because the
 *      data already lives in the currently BOUND buffer.
 *   5. Clean up: disable state, unbind, and delete the buffer.
 */

GLuint VBOid;
```

```
void triangle() {
    // 1. initialize the triangle using an array
    GLfloat trianglevertices[] = {
        0.0f, .75f, 0.0f,
        -0.75f, 0.0f, 0.0f,
        0.75f, 0.0f, 0.0f
    };

    // 2. generate a Vertex Buffer Object ID and bind it
    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, trianglevertices,
GL_STATIC_DRAW);

    // 3. activate and specify pointer to vertex array
    glEnableClientState(GL_VERTEX_ARRAY);

    // 4. draw the primitive object (0 = "use the currently bound buffer")
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_TRIANGLES, 0, 3);

    // 5. disable client side capability for normal operation
    glDisableClientState(GL_VERTEX_ARRAY);

    // bind with 0, so switch back to normal pointer operation
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor4f(.16f, .72f, .08f, 1.0f);
    triangle();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE_
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
```

```
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 03 — GL_POINTS From a VBO

Concept: Applying the 5-step VBO pattern to GL_POINTS.

Expected output: Four large white points, one at each corner of a square, sourced from GPU memory.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 03 - GL_POINTS as a VBO
 * ------------------------------------------------------------
 * Concept: The same 5-step pattern applied to GL_POINTS.
 */

GLuint VBOid;

void points() {
    glPointSize(15.0f);

    GLfloat pointVertices[] = {
        -0.75f, -0.75f, 0.0f,
        -0.75f,  0.75f, 0.0f,
         0.75f,  0.75f, 0.0f,
         0.75f, -0.75f, 0.0f
    };

    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 12, pointVertices, GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_POINTS, 0, 4);
```

```
    glDisableClientState(GL_VERTEX_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);
    points();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE__
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 04 — GL_LINES From a VBO

Concept: A two-vertex VBO rendered with GL_LINES.

Expected output: A single thick horizontal yellow line.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
```

```
using namespace std;

/*
 * Example 04 - GL_LINES as a VBO
 * ------------------------------------------------------------
 * Concept: The same 5-step pattern applied to GL_LINES.
 */

GLuint VBOid;

void line() {
    GLfloat lineVertices[] = {
        -0.8f, 0.0f, 0.0f,
         0.8f, 0.0f, 0.0f
    };

    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 6, lineVertices, GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glLineWidth(4.0f);
    glDrawArrays(GL_LINES, 0, 2);
    glDisableClientState(GL_VERTEX_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 0.0f);
    line();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE__
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
```

```
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 05 — GL_QUADS From a VBO

Concept: A 4-vertex VBO rendered with GL_QUADS.

Expected output: A solid blue filled square.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 05 - GL_QUADS as a VBO
 * ------------------------------------------------------------
 * Concept: The same 5-step pattern applied to GL_QUADS.
 */

GLuint VBOid;

void square() {
    GLfloat quadVertices[] = {
        -0.5f, -0.5f, 0.0f,
         0.5f, -0.5f, 0.0f,
         0.5f,  0.5f, 0.0f,
        -0.5f,  0.5f, 0.0f
    };

    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 12, quadVertices, GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_QUADS, 0, 4);
    glDisableClientState(GL_VERTEX_ARRAY);
```

```
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.2f, 0.5f, 1.0f);
    square();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE__
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 06 — GL_POLYGON From a VBO

Concept: A 5-vertex VBO rendered with GL_POLYGON.

Expected output: A solid orange filled pentagon.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;
```

```
/*
 * Example 06 - GL_POLYGON as a VBO
 * ------------------------------------------------------------
 * Concept: The same 5-step pattern applied to GL_POLYGON.
 */

GLuint VBOid;

void pentagon() {
    GLfloat pentagonVertices[] = {
         0.0f,  0.75f, 0.0f,
        -0.71f, 0.23f, 0.0f,
        -0.44f, -0.61f, 0.0f,
         0.44f, -0.61f, 0.0f,
         0.71f, 0.23f, 0.0f
    };

    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 15, pentagonVertices,
GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_POLYGON, 0, 5);
    glDisableClientState(GL_VERTEX_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 0.6f, 0.0f);
    pentagon();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE_
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
```

```
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 07 — glBufferData Usage Hints Compared

Concept: The same triangle shape drawn three times with GL_STATIC_DRAW, GL_DYNAMIC_DRAW, and GL_STREAM_DRAW, to compare the usage-hint argument.

Expected output: Three triangles side by side, each built with a different glBufferData usage hint.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 07 - glBufferData Usage Hints
 * ------------------------------------------------------------
 * Concept: glBufferData's final argument is a USAGE HINT telling
 * OpenGL how the data will be used, so the driver can choose the best
 * memory location/strategy:
 *   GL_STATIC_DRAW  - set once, drawn many times (most common for
 *                     unchanging geometry).
 *   GL_DYNAMIC_DRAW - changed repeatedly, drawn many times (used
 *                     later in this module for animated VBOs).
 *   GL_STREAM_DRAW  - set once, drawn only a few times.
 * The RENDERED RESULT is identical either way - only the performance
 * characteristics differ. This example draws the same triangle three
 * times, each from a separately created VBO using a different hint,
 * purely to show the syntax side by side.
 */

GLuint staticVBO, dynamicVBO, streamVBO;

GLuint makeTriangleVBO(GLfloat offsetX, GLenum usage) {
    GLfloat verts[] = {
        offsetX + 0.0f, 0.75f, 0.0f,
        offsetX - 0.2f, 0.55f, 0.0f,
```

```
        offsetX + 0.2f, 0.55f, 0.0f
    };
    GLuint id;
    glGenBuffers(1, &id);
    glBindBuffer(GL_ARRAY_BUFFER, id);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, verts, usage);
    return id;
}

void drawVBO(GLuint id) {
    glBindBuffer(GL_ARRAY_BUFFER, id);
    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    staticVBO = makeTriangleVBO(-0.6f, GL_STATIC_DRAW);
    dynamicVBO = makeTriangleVBO(0.0f, GL_DYNAMIC_DRAW);
    streamVBO = makeTriangleVBO(0.6f, GL_STREAM_DRAW);

    glColor3f(0.9f, 0.3f, 0.3f);
    drawVBO(staticVBO);
    glColor3f(0.3f, 0.9f, 0.3f);
    drawVBO(dynamicVBO);
    glColor3f(0.3f, 0.3f, 0.9f);
    drawVBO(streamVBO);

    glDeleteBuffers(1, &staticVBO);
    glDeleteBuffers(1, &dynamicVBO);
    glDeleteBuffers(1, &streamVBO);

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE__
    GLenum err = glewInit();
    if (err == GLEW_OK) {
```

```
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 08 — Two Independent VBOs in One Program

Concept: Managing two separate GLuint buffer IDs, each with its own glGenBuffers/glBindBuffer/glBufferData sequence.

Expected output: Two separate triangles, each drawn from its own independent VBO.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 08 - Two Separate VBOs, Two Separate Shapes
 * ------------------------------------------------------------
 * Concept: An application is not limited to one VBO. Each shape can
 * have its own buffer ID, generated and bound independently, and
 * drawn in its own turn.
 */

GLuint leftVBO, rightVBO;

void triangleLeft() {
    GLfloat verts[] = { -0.7f, 0.4f, 0.0f, -0.9f, -0.2f, 0.0f, -0.5f, -0.2f, 0.0f };
    glGenBuffers(1, &leftVBO);
    glBindBuffer(GL_ARRAY_BUFFER, leftVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, verts, GL_STATIC_DRAW);
    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColor3f(1.0f, 0.4f, 0.1f);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
}
```

```
void triangleRight() {
    GLfloat verts[] = { 0.7f, 0.4f, 0.0f, 0.5f, -0.2f, 0.0f, 0.9f, -0.2f, 0.0f };
    glGenBuffers(1, &rightVBO);
    glBindBuffer(GL_ARRAY_BUFFER, rightVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, verts, GL_STATIC_DRAW);
    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColor3f(0.1f, 0.4f, 1.0f);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    triangleLeft();
    triangleRight();
    glDeleteBuffers(1, &leftVBO);
    glDeleteBuffers(1, &rightVBO);
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE__
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 09 — glDrawElements(): A VBO Plus an Element Buffer

Concept: The same triangle as Example 02, this time using a GL_ELEMENT_ARRAY_BUFFER index buffer and glDrawElements with a byte OFFSET.

Expected output: The same green triangle, drawn through the GPU-resident indexed-rendering path.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 09 - A Second Buffer for Indices (GL_ELEMENT_ARRAY_BUFFER)
 * ------------------------------------------------------------
 * Concept: Just as GL_ARRAY_BUFFER holds vertex data on the GPU,
 * GL_ELEMENT_ARRAY_BUFFER holds INDEX data on the GPU. The pattern is
 * identical: glGenBuffers, glBindBuffer, glBufferData - just with a
 * different target and a GLubyte index array. Once bound,
 * glDrawElements's final argument becomes an OFFSET (0) instead of a
 * real pointer, exactly like glVertexPointer's did for GL_ARRAY_BUFFER.
 */

GLuint VBOid, VBOindex;

void triangle() {
    GLfloat trianglevertices[] = {
        0.0f, .75f, 0.0f,
        -0.75f, 0.0f, 0.0f,
        0.75f, 0.0f, 0.0f
    };
    GLubyte indices[] = { 0, 1, 2 };

    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, trianglevertices,
GL_STATIC_DRAW);

    glGenBuffers(1, &VBOindex);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, VBOindex);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(GLubyte) * 3, indices, GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawElements(GL_TRIANGLES, 3, GL_UNSIGNED_BYTE, 0);
    glDisableClientState(GL_VERTEX_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);
    glDeleteBuffers(1, &VBOindex);
}
```

```
void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor4f(.16f, .72f, .08f, 1.0f);
    triangle();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE_
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 10 — glDrawElements(): Reusing Shared Vertices on the GPU

Concept: A quad built from 4 unique GPU-resident vertices plus a 6-entry GPU-resident index buffer.

Expected output: A solid orange filled square, built from only 4 stored vertices in GPU memory.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 10 - Index Buffer Reusing Shared Vertices (Quad from 2 Triangles)
 * ------------------------------------------------------------
```

```
 * Concept: Combines Module 4's shared-vertex idea with GPU-resident
 * buffers. A quad needs only 4 UNIQUE vertices stored on the GPU; the
 * index buffer (also GPU-resident) tells OpenGL how to connect them
 * into 2 triangles.
 */

GLuint VBOid, VBOindex;

void quad() {
    GLfloat quadVertices[] = {
        -0.5f, -0.5f, 0.0f,  // 0
         0.5f, -0.5f, 0.0f,  // 1
         0.5f,  0.5f, 0.0f,  // 2
        -0.5f,  0.5f, 0.0f   // 3
    };
    GLubyte indices[] = { 0, 1, 2, 0, 2, 3 };

    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 12, quadVertices, GL_STATIC_DRAW);

    glGenBuffers(1, &VBOindex);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, VBOindex);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(GLubyte) * 6, indices, GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_BYTE, 0);
    glDisableClientState(GL_VERTEX_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);
    glDeleteBuffers(1, &VBOindex);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.9f, 0.5f, 0.1f);
    quad();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);
```

```
#ifndef __APPLE__
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 11 — Position VBO + Color VBO Together

Concept: Two separate buffers — one for position, one for color — bound and pointed to independently before one glDrawArrays call.

Expected output: A triangle with red, green, and blue corners, smoothly blending across its interior.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 11 - A Second VBO for Per-Vertex Color
 * ------------------------------------------------------------
 * Concept: Exactly as position data can live in a GPU-resident VBO,
 * so can color data. TWO separate buffers are generated, bound, and
 * filled in turn; both GL_VERTEX_ARRAY and GL_COLOR_ARRAY client
 * states are enabled together for the draw call.
 */

GLuint posVBO, colorVBO;

void coloredTriangle() {
    GLfloat positions[] = {
        0.0f, 0.75f, 0.0f,
        -0.75f, 0.0f, 0.0f,
        0.75f, 0.0f, 0.0f
    };
    GLfloat colors[] = {
```

```
        1.0f, 0.0f, 0.0f,
        0.0f, 1.0f, 0.0f,
        0.0f, 0.0f, 1.0f
    };

    glGenBuffers(1, &posVBO);
    glBindBuffer(GL_ARRAY_BUFFER, posVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, positions, GL_STATIC_DRAW);

    glGenBuffers(1, &colorVBO);
    glBindBuffer(GL_ARRAY_BUFFER, colorVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, colors, GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, posVBO);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glBindBuffer(GL_ARRAY_BUFFER, colorVBO);
    glColorPointer(3, GL_FLOAT, 0, 0);

    glDrawArrays(GL_TRIANGLES, 0, 3);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &posVBO);
    glDeleteBuffers(1, &colorVBO);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    coloredTriangle();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE_
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
```

```
    glutMainLoop();
#endif
    return 0;
}
```

Example 12 — One Interleaved VBO (Position + Color)

Concept: Position and color interleaved into a SINGLE buffer, read out with glVertexPointer/glColorPointer using a shared stride and small byte-offset casts.

Expected output: A red/green/blue triangle, identical in appearance to Example 11, sourced from one interleaved VBO.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 12 - One VBO, Interleaved Position + Color
 * ------------------------------------------------------------
 * Concept: Position and color can share ONE buffer, interleaved as
 * [x,y,z,r,g,b, x,y,z,r,g,b, ...]. glVertexPointer/glColorPointer use
 * a non-zero stride and a small BYTE OFFSET (cast to a pointer) to
 * read their own fields from the same bound buffer.
 */

GLuint VBOid;

void interleavedTriangle() {
    GLfloat data[] = {
        // x,     y,     z,     r,    g,    b
        0.0f,  0.75f, 0.0f,  1.0f, 0.0f, 0.0f,
        -0.75f, 0.0f, 0.0f,  0.0f, 1.0f, 0.0f,
        0.75f,  0.0f, 0.0f,  0.0f, 0.0f, 1.0f
    };
    GLsizei stride = 6 * sizeof(GLfloat);

    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(data), data, GL_STATIC_DRAW);
```

```
    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glVertexPointer(3, GL_FLOAT, stride, (void*)0);
    glColorPointer(3, GL_FLOAT, stride, (void*)(3 * sizeof(GLfloat)));

    glDrawArrays(GL_TRIANGLES, 0, 3);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    interleavedTriangle();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE__
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 13 — VBO Pinwheel: Position + Color Buffers

Concept: Module 4's four-triangle pinwheel rebuilt so its position and color data both live in GPU-resident VBOs.

Expected output: A four-armed pinwheel, each arm fading red-green-blue, now rendered entirely from VBOs.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 13 - The Colored Pinwheel, Rendered From VBOs
 * ------------------------------------------------------------
 * Concept: Module 4's four-triangle pinwheel scene, rebuilt so BOTH
 * the position data and the color data live in GPU-resident VBOs
 * instead of plain client-side arrays.
 */

GLuint posVBO, colorVBO;

void pinwheel() {
    GLfloat positions[] = {
        0.0f, 0.0f, 0.0f,    -0.5f, 0.10f, 0.0f,   -0.5f, -0.10f, 0.0f,
        0.0f, 0.0f, 0.0f,     0.5f, 0.10f, 0.0f,    0.5f, -0.10f, 0.0f,
        0.0f, 0.0f, 0.0f,    -0.10f, 0.50f, 0.0f,   0.10f,  0.50f, 0.0f,
        0.0f, 0.0f, 0.0f,    -0.10f, -0.50f, 0.0f,  0.10f, -0.50f, 0.0f
    };
    GLfloat colors[] = {
        1.0f,0.0f,0.0f,  0.0f,1.0f,0.0f,  0.0f,0.0f,1.0f,
        1.0f,0.0f,0.0f,  0.0f,1.0f,0.0f,  0.0f,0.0f,1.0f,
        1.0f,0.0f,0.0f,  0.0f,1.0f,0.0f,  0.0f,0.0f,1.0f,
        1.0f,0.0f,0.0f,  0.0f,1.0f,0.0f,  0.0f,0.0f,1.0f
    };

    glGenBuffers(1, &posVBO);
    glBindBuffer(GL_ARRAY_BUFFER, posVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(positions), positions, GL_STATIC_DRAW);

    glGenBuffers(1, &colorVBO);
    glBindBuffer(GL_ARRAY_BUFFER, colorVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(colors), colors, GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, posVBO);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glBindBuffer(GL_ARRAY_BUFFER, colorVBO);
    glColorPointer(3, GL_FLOAT, 0, 0);
```

```
    glDrawArrays(GL_TRIANGLES, 0, 12);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &posVBO);
    glDeleteBuffers(1, &colorVBO);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    pinwheel();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE_
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 14 — glMapBuffer(): Editing GPU Data From the CPU

Concept: glMapBuffer maps a VBO's GPU memory into a client-accessible pointer; the program edits one vertex, then calls glUnmapBuffer before drawing.

Expected output: A triangle whose top vertex has been nudged upward compared to its original position, edited directly in GPU memory.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
```

```
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 14 - glMapBuffer(): Editing VBO Data Already on the GPU
 * ---------------------------------------------------------·
 * Concept: Once vertex data has been uploaded to the GPU, editing the
 * ORIGINAL client-side array no longer changes what is drawn - the
 * GPU has its own copy. glMapBuffer(target, access) hands back a
 * pointer that lets the application read/write that GPU-resident data
 * directly; glUnmapBuffer(target) must be called afterward to release
 * the mapping. Here, a single glMapBuffer call nudges the triangle's
 * three vertices to the right by a fixed amount, once, when the
 * program starts.
 */

GLuint VBOid;

void triangle() {
    GLfloat trianglevertices[] = {
        0.0f, .75f, 0.0f,
        -0.75f, 0.0f, 0.0f,
        0.75f, 0.0f, 0.0f
    };

    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, trianglevertices,
GL_DYNAMIC_DRAW);

    // Map the buffer, nudge every vertex's X coordinate, then unmap
    GLfloat* mapped = (GLfloat*) glMapBuffer(GL_ARRAY_BUFFER, GL_READ_WRITE);
    mapped[0] += 0.1f;  // vertex 0's x
    mapped[3] += 0.1f;  // vertex 1's x
    mapped[6] += 0.1f;  // vertex 2's x
    glUnmapBuffer(GL_ARRAY_BUFFER);

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
```

```
    glColor4f(.16f, .72f, .08f, 1.0f);
    triangle();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE_
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 15 — ESC-Key Cleanup and Exit

Concept: A monitorKeyboard() handler that releases all VBOs with glDeleteBuffers before calling exit(0) when ESC (key 27) is pressed.

Expected output: A simple shape that renders normally and cleans up its GPU buffers when you press ESC to quit.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 15 - Proper VBO Cleanup on Exit (ESC Key)
 * ------------------------------------------------------------
```

```
 * Concept: A VBO occupies GPU memory for as long as the buffer ID is
 * alive. A well-behaved program releases that memory before exiting.
 * Pressing ESC here disables the client state, unbinds the buffer,
 * deletes it with glDeleteBuffers, and then exits - mirroring the
 * lecture's monitorKeyboard() cleanup pattern.
 */

GLuint VBOid;

void triangle() {
    GLfloat trianglevertices[] = {
        0.0f, .75f, 0.0f,
        -0.75f, 0.0f, 0.0f,
        0.75f, 0.0f, 0.0f
    };

    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, trianglevertices,
GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor4f(.16f, .72f, .08f, 1.0f);
    triangle();
    glFlush();
}

void monitorKeyboard(unsigned char key, int x, int y) {
    if (key == 27) {   // ESC
        // Disable client side capability for normal operation
        glDisableClientState(GL_VERTEX_ARRAY);

        // bind with 0, so switch back to normal pointer operation
        glBindBuffer(GL_ARRAY_BUFFER, 0);
        glDeleteBuffers(1, &VBOid);

        exit(0);
    }
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
```

```
    glutCreateWindow("simple");

    glutDisplayFunc(display);
    glutKeyboardFunc(monitorKeyboard);

#ifndef __APPLE_
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 16 — Continuous Animation via glutTimerFunc + glMapBuffer

Concept: A self-rescheduling glutTimerFunc callback that maps the VBO, shifts a vertex, unmaps it, and calls glutPostRedisplay every second.

Expected output: A triangle whose shape visibly changes once per second, driven by repeated glMapBuffer edits.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 16 - Continuous VBO Animation via glMapBuffer + Timer
 * ------------------------------------------------------------
 * Concept: Combines glutTimerFunc (Module 3) with glMapBuffer to
 * animate a shape that lives ENTIRELY in GPU memory. Every second, the
 * timer callback maps the buffer, nudges the triangle sideways, and
 * unmaps it - the same pattern as the lecture's modifyTriangle(),
 * running continuously via a self-rescheduling timer.
 */

GLuint VBOid;
bool triangleReady = false;
```

```
void triangle() {
    if (!triangleReady) {
        GLfloat trianglevertices[] = {
            0.0f, .75f, 0.0f,
            -0.75f, 0.0f, 0.0f,
            0.75f, 0.0f, 0.0f
        };
        glGenBuffers(1, &VBOid);
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
        glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, trianglevertices,
GL_DYNAMIC_DRAW);
        triangleReady = true;
    } else {
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    }

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor4f(.16f, .72f, .08f, 1.0f);
    triangle();
    glFlush();
}

void animateStep(int value) {
    static float dx = 0.02f;

    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    GLfloat* mapped = (GLfloat*) glMapBuffer(GL_ARRAY_BUFFER, GL_READ_WRITE);
    if (mapped[0] > 0.3f || mapped[0] < -0.3f) dx = -dx;
    mapped[0] += dx;
    mapped[3] += dx;
    mapped[6] += dx;
    glUnmapBuffer(GL_ARRAY_BUFFER);

    glutPostRedisplay();
    glutTimerFunc(200, animateStep, 0);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);
```

```
    glutTimerFunc(200, animateStep, 0);

#ifndef __APPLE_
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 17 — Indexed Drawing With BOTH a Color VBO and an Element VBO

Concept: A quad combining a position VBO, a color VBO, and a GL_ELEMENT_ARRAY_BUFFER index buffer, all drawn with one glDrawElements call.

Expected output: A smoothly shaded quad with four different corner colors, built entirely from three GPU-resident buffers.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 17 - Indexed Drawing With BOTH a Color VBO and an Element VBO
 * ------------------------------------------------------------
 * Concept: Combines Example 10's shared-vertex index buffer with
 * Example 11's separate color buffer. A quad is described by 4 unique
 * positions, 4 unique colors, and one 6-entry index buffer -
 * everything GPU-resident, everything drawn with ONE glDrawElements call.
 */

GLuint posVBO, colorVBO, indexVBO;

void coloredIndexedQuad() {
    GLfloat positions[] = {
        -0.5f, -0.5f, 0.0f,
         0.5f, -0.5f, 0.0f,
         0.5f,  0.5f, 0.0f,
```

```
        -0.5f,  0.5f, 0.0f
    };
    GLfloat colors[] = {
        1.0f, 0.0f, 0.0f,
        0.0f, 1.0f, 0.0f,
        0.0f, 0.0f, 1.0f,
        1.0f, 1.0f, 0.0f
    };
    GLubyte indices[] = { 0, 1, 2, 0, 2, 3 };

    glGenBuffers(1, &posVBO);
    glBindBuffer(GL_ARRAY_BUFFER, posVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(positions), positions, GL_STATIC_DRAW);

    glGenBuffers(1, &colorVBO);
    glBindBuffer(GL_ARRAY_BUFFER, colorVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(colors), colors, GL_STATIC_DRAW);

    glGenBuffers(1, &indexVBO);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, indexVBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices, GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, posVBO);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glBindBuffer(GL_ARRAY_BUFFER, colorVBO);
    glColorPointer(3, GL_FLOAT, 0, 0);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, indexVBO);

    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_BYTE, 0);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &posVBO);
    glDeleteBuffers(1, &colorVBO);
    glDeleteBuffers(1, &indexVBO);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    coloredIndexedQuad();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
```

```
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE__
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 18 — Multi-Shape Scene From Three Independent VBOs

Concept: A small scene (roof, wall, sun) where each shape owns its own VBO, all drawn in the same display() call.

Expected output: A simple house shape with a triangular roof, a rectangular wall, and a small sun, each sourced from a separate VBO.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 18 - A Composite Scene, Every Shape a Separate VBO
 * ----------------------------------------------------------
 * Concept: A small scene combining three independently managed VBOs -
 * a triangle "roof", a quad "wall", and a GL_POLYGON "sun" - each
 * generated, bound, drawn, and released in its own turn.
 */

GLuint roofVBO, wallVBO, sunVBO;

void roof() {
```

```
    GLfloat v[] = { 0.0f, 0.6f, 0.0f, -0.5f, 0.1f, 0.0f, 0.5f, 0.1f, 0.0f };
    glGenBuffers(1, &roofVBO);
    glBindBuffer(GL_ARRAY_BUFFER, roofVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(v), v, GL_STATIC_DRAW);
    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColor3f(0.7f, 0.2f, 0.2f);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
}

void wall() {
    GLfloat v[] = { -0.5f, 0.1f, 0.0f,  0.5f, 0.1f, 0.0f,  0.5f, -0.5f, 0.0f,  -0.5f, -
0.5f, 0.0f };
    glGenBuffers(1, &wallVBO);
    glBindBuffer(GL_ARRAY_BUFFER, wallVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(v), v, GL_STATIC_DRAW);
    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColor3f(0.85f, 0.7f, 0.5f);
    glDrawArrays(GL_QUADS, 0, 4);
    glDisableClientState(GL_VERTEX_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
}

void sun() {
    GLfloat v[] = {
        0.75f, 0.7f, 0.0f, 0.65f, 0.6f, 0.0f, 0.75f, 0.55f, 0.0f,
        0.85f, 0.6f, 0.0f, 0.8f, 0.72f, 0.0f
    };
    glGenBuffers(1, &sunVBO);
    glBindBuffer(GL_ARRAY_BUFFER, sunVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(v), v, GL_STATIC_DRAW);
    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColor3f(1.0f, 0.85f, 0.1f);
    glDrawArrays(GL_POLYGON, 0, 5);
    glDisableClientState(GL_VERTEX_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    wall();
    roof();
    sun();
    glDeleteBuffers(1, &roofVBO);
    glDeleteBuffers(1, &wallVBO);
    glDeleteBuffers(1, &sunVBO);
    glFlush();
}
```

```
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE__
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 19 — Batching Many Shapes Into One VBO

Concept: 50 small triangles generated procedurally with sinf() and uploaded into ONE buffer, rendered with a single glDrawArrays call.

Expected output: A ring or scattering of 50 small colored triangles, all drawn from one buffer in one draw call.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 19 - Batching Many Triangles Into ONE VBO
 * ------------------------------------------------------------
 * Concept: The real-world payoff of VBOs shows up at scale. Here, 50
 * small triangles are generated procedurally into ONE large array,
 * uploaded to the GPU in ONE glBufferData call, and rendered with ONE
 * glDrawArrays call - instead of 50 separate immediate-mode
```

```
 * glBegin/glEnd blocks or 50 separate small VBOs.
 */

const int TRIANGLE_COUNT = 50;
GLuint VBOid;

void batch() {
    GLfloat verts[TRIANGLE_COUNT * 9];
    for (int i = 0; i < TRIANGLE_COUNT; i++) {
        float t = (float)i / TRIANGLE_COUNT;
        float cx = -0.9f + t * 1.8f;
        float cy = 0.3f * sinf(t * 6.28f * 2.0f);
        int b = i * 9;
        verts[b + 0] = cx;        verts[b + 1] = cy + 0.03f; verts[b + 2] = 0.0f;
        verts[b + 3] = cx - 0.02f; verts[b + 4] = cy - 0.02f; verts[b + 5] = 0.0f;
        verts[b + 6] = cx + 0.02f; verts[b + 7] = cy - 0.02f; verts[b + 8] = 0.0f;
    }

    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(verts), verts, GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColor3f(0.2f, 0.8f, 0.9f);
    glDrawArrays(GL_TRIANGLES, 0, TRIANGLE_COUNT * 3);
    glDisableClientState(GL_VERTEX_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    batch();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);


#ifndef __APPLE__
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
```

```
    }
#else
    glutMainLoop();
#endif
    return 0;
}
```

Example 20 — Capstone: Animated VBO Scene

Concept: A position+color VBO triangle animated every second via glMapBuffer/glutTimerFunc, plus a static indexed ground quad, plus ESC-key cleanup releasing all four buffers.

Expected output: A colored triangle that shifts position once per second above a static ground quad; pressing ESC releases all GPU buffers and exits.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

/*
 * Example 20 - Capstone: A Full VBO-Based, Animated, Cleaned-Up Scene
 * ------------------------------------------------------------
 * Concept: Combines every technique from this module into one
 * program:
 *   - A position VBO and a color VBO for a triangle (Example 11).
 *   - An element (index) VBO for a quad "ground" (Example 10).
 *   - Continuous glMapBuffer-driven animation via a self-rescheduling
 *     glutTimerFunc (Example 16).
 *   - Proper GPU memory cleanup on ESC (Example 15).
 */

GLuint triPosVBO, triColorVBO, groundVBO, groundIndexVBO;
bool ready = false;

void buildScene() {
    GLfloat triPos[] = {
        0.0f, 0.75f, 0.0f,
        -0.3f, 0.35f, 0.0f,
        0.3f, 0.35f, 0.0f
    };
    GLfloat triColor[] = {
```

```
        1.0f, 0.2f, 0.2f,
        0.2f, 1.0f, 0.2f,
        0.2f, 0.2f, 1.0f
    };
    GLfloat groundVerts[] = {
        -0.9f, -0.9f, 0.0f,  0.9f, -0.9f, 0.0f,  0.9f, -0.6f, 0.0f,  -0.9f, -0.6f, 0.0f
    };
    GLubyte groundIndices[] = { 0, 1, 2, 0, 2, 3 };

    glGenBuffers(1, &triPosVBO);
    glBindBuffer(GL_ARRAY_BUFFER, triPosVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(triPos), triPos, GL_DYNAMIC_DRAW);

    glGenBuffers(1, &triColorVBO);
    glBindBuffer(GL_ARRAY_BUFFER, triColorVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(triColor), triColor, GL_STATIC_DRAW);

    glGenBuffers(1, &groundVBO);
    glBindBuffer(GL_ARRAY_BUFFER, groundVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(groundVerts), groundVerts, GL_STATIC_DRAW);

    glGenBuffers(1, &groundIndexVBO);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, groundIndexVBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(groundIndices), groundIndices,
GL_STATIC_DRAW);

    ready = true;
}

void drawGround() {
    glEnableClientState(GL_VERTEX_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, groundVBO);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, groundIndexVBO);
    glColor3f(0.3f, 0.6f, 0.3f);
    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_BYTE, 0);
    glDisableClientState(GL_VERTEX_ARRAY);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);
}

void drawTriangle() {
    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, triPosVBO);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glBindBuffer(GL_ARRAY_BUFFER, triColorVBO);
    glColorPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
}
```

```
void display() {
    if (!ready) buildScene();
    glClear(GL_COLOR_BUFFER_BIT);
    drawGround();
    drawTriangle();
    glFlush();
}

void animateStep(int value) {
    static float dy = -0.01f;

    glBindBuffer(GL_ARRAY_BUFFER, triPosVBO);
    GLfloat* mapped = (GLfloat*) glMapBuffer(GL_ARRAY_BUFFER, GL_READ_WRITE);
    if (mapped[1] > 0.75f || mapped[1] < 0.4f) dy = -dy;
    mapped[1] += dy; mapped[4] += dy; mapped[7] += dy;
    glUnmapBuffer(GL_ARRAY_BUFFER);
    glBindBuffer(GL_ARRAY_BUFFER, 0);

    glutPostRedisplay();
    glutTimerFunc(100, animateStep, 0);
}

void monitorKeyboard(unsigned char key, int x, int y) {
    if (key == 27) {   // ESC: release every GPU buffer before exiting
        glBindBuffer(GL_ARRAY_BUFFER, 0);
        glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);
        glDeleteBuffers(1, &triPosVBO);
        glDeleteBuffers(1, &triColorVBO);
        glDeleteBuffers(1, &groundVBO);
        glDeleteBuffers(1, &groundIndexVBO);
        exit(0);
    }
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("simple");

    glutDisplayFunc(display);
    glutKeyboardFunc(monitorKeyboard);
    glutTimerFunc(100, animateStep, 0);

#ifndef __APPLE__
    GLenum err = glewInit();
    if (err == GLEW_OK) {
        glutMainLoop();
    }
#else
    glutMainLoop();
```

```
#endif
    return 0;
}
```

Part B — Practice Exercises (Q01–Q20)

Attempt each exercise on your own first. Compile and run your own solution, compare its output to the task description, and only THEN check Part C. Difficulty is marked Easy, Medium, or Hard. Each exercise's "Task" line is the exact question you must answer with your program.

Easy Exercises

Exercise Q01 — VBO X-Pattern Points [Easy]

Objective: Practice converting GL_POINTS to a VBO using the 5-step pattern.

Task (Question): Convert an immediate-mode GL_POINTS block (5 points forming an X pattern with a center point) into a VBO, using all 5 steps (generate, bind, upload, point-and-draw with an offset, clean up).

Hints:

- Set glPointSize BEFORE glDrawArrays, just as you would before glBegin.
- Once a buffer is bound, the last argument to glVertexPointer is a byte OFFSET (usually 0), not a real pointer.

Exercise Q02 — VBO Line Loop Square [Easy]

Objective: Practice GL_LINE_LOOP sourced from a VBO instead of GL_QUADS.

Task (Question): Draw the outline of a square using GL_LINE_LOOP via a VBO, with 4 unique vertices of your own choosing.

Hints:

- GL_LINE_LOOP automatically connects the last vertex back to the first — you do not need to repeat the first vertex.
- glLineWidth (called before the draw) makes the outline easier to see.

Exercise Q03 — Static Hexagon With the Correct Usage Hint [Easy]

Objective: Practice choosing and justifying a glBufferData usage hint for data that never changes.

Task (Question): Draw a filled, six-sided hexagon using GL_POLYGON via a VBO, choosing the usage hint that best matches data uploaded once and drawn many times, and add a code comment explaining WHY you chose it.

Hints:

- Data set once and used many times without modification matches GL_STATIC_DRAW.

- Plan your six (x, y) points on paper first, going consistently clockwise or counter-clockwise.

Exercise Q04 — Full Cleanup Checklist [Easy]

Objective: Practice the complete VBO cleanup sequence (disable, unbind, delete).

Task (Question): Draw any simple shape from a VBO, and BEFORE the program exits, perform the full cleanup checklist: glDisableClientState, glBindBuffer(target, 0), and glDeleteBuffers, with a comment above each line explaining what it undoes.

Hints:

- Unbinding the buffer (binding 0) should happen before deleting it.
- glDeleteBuffers takes the ADDRESS of your GLuint ID variable, the same as glGenBuffers.

Exercise Q05 — Two Separate VBOs Side by Side [Easy]

Objective: Practice managing two independent GLuint buffer IDs in the same program.

Task (Question): Draw two triangles side by side, each one sourced from its OWN separate VBO (two GLuint IDs, two glGenBuffers/glBindBuffer/glBufferData sequences).

Hints:

- Give the two GLuint variables clearly different names so you don't mix up which buffer is bound when.
- Re-bind the correct buffer immediately before each glDrawArrays call.

Exercise Q06 — Flat-Colored VBO Quad [Easy]

Objective: Practice a basic VBO conversion for GL_QUADS with a single flat color.

Task (Question): Convert an immediate-mode filled square into a VBO using GL_QUADS, keeping a single flat glColor3f color for the whole shape (no color array needed yet).

Hints:

- You only need ONE VBO here — position data only.
- Call glColor3f once, before the draw call, exactly as you would in immediate mode.

Exercise Q07 — VBO Line-Strip Zigzag [Easy]

Objective: Practice GL_LINE_STRIP sourced from a VBO with more than 2 vertices.

Task (Question): Draw a zigzag line (at least 5 vertices) using GL_LINE_STRIP via a VBO.

Hints:

- GL_LINE_STRIP connects each vertex to the next, but does NOT close back to the first (unlike GL_LINE_LOOP).
- Alternate the y-coordinate up and down as x increases to create the zigzag pattern.

Medium Exercises

Exercise Q08 — Indexed Quad: Position VBO + Separate Index VBO [Medium]

Objective: Practice combining a position VBO with a GL_ELEMENT_ARRAY_BUFFER index VBO.

Task (Question): Build a quad from 4 unique GPU-resident vertices plus a 6-entry GPU-resident index buffer, drawn with glDrawElements.

Hints:

- You need TWO buffer IDs: one bound to GL_ARRAY_BUFFER, one bound to GL_ELEMENT_ARRAY_BUFFER.
- The index array's data type (GLubyte, GLuint, etc.) must match the `type` argument you pass to glDrawElements.

Exercise Q09 — Two-VBO Colored Triangle [Medium]

Objective: Practice combining a position VBO and a SEPARATE color VBO (not interleaved).

Task (Question): Draw a triangle with a different color at each vertex, using a position VBO and a separate color VBO (two independent buffers, not one interleaved buffer).

Hints:

- Both GL_VERTEX_ARRAY and GL_COLOR_ARRAY must be enabled, and each needs its own buffer bound right before its pointer call.
- Bind the position buffer and call glVertexPointer, THEN bind the color buffer and call glColorPointer.

Exercise Q10 — Interleaved VBO Quad [Medium]

Objective: Practice building and reading an interleaved position+color VBO with a non-zero stride.

Task (Question): Build a single interleaved VBO holding both position and color data for a quad (like Example 12), and render it using stride-based glVertexPointer/glColorPointer calls.

Hints:

- Each vertex's 6 floats are x,y,z,r,g,b in that order — stride is 6 * sizeof(GLfloat).
- The color pointer's byte-offset argument must skip past the 3 position floats: (void*)(3 * sizeof(GLfloat)).

Exercise Q11 — One-Shot glMapBuffer Vertical Shift [Medium]

Objective: Practice mapping a VBO, editing it once, and unmapping it before drawing.

Task (Question): Draw a triangle from a VBO, then use glMapBuffer(GL_ARRAY_BUFFER, GL_READ_WRITE) to shift ONE vertex upward by editing the mapped memory directly, calling glUnmapBuffer before the draw call.

Hints:

- glMapBuffer returns a void* — cast it to GLfloat* before indexing into it.
- ALWAYS call glUnmapBuffer immediately after you finish editing, before any draw call uses that buffer.

Exercise Q12 — Timer + glMapBuffer Moving Point [Medium]

Objective: Practice a glutTimerFunc callback that repeatedly maps and edits a VBO.

Task (Question): Animate a single GL_POINTS vertex so it moves one step to the right every second, using a glutTimerFunc callback that maps the VBO, updates the x-coordinate, unmaps it, and reschedules itself.

Hints:

- Call glutPostRedisplay() at the end of your timer callback so the change actually appears on screen.
- Reschedule the timer by calling glutTimerFunc again as the LAST line inside the callback itself.

Exercise Q13 — ESC Cleanup Releasing Two VBOs [Medium]

Objective: Practice cleanup discipline when a program owns MORE than one buffer.

Task (Question): Draw a scene using two separate VBOs, and write a monitorKeyboard() handler that releases BOTH buffers with glDeleteBuffers before exit(0) when ESC is pressed.

Hints:

- Delete each buffer by its own GLuint ID — one glDeleteBuffers call per buffer, or one call with an array of both IDs.
- Register your keyboard handler with glutKeyboardFunc inside main(), before glutMainLoop().

Exercise Q14 — Indexed GL_TRIANGLE_FAN Pentagon [Medium]

Objective: Practice glDrawElements with GL_TRIANGLE_FAN and a shared center vertex, entirely on the GPU.

Task (Question): Build a pentagon from GL_TRIANGLE_FAN using glDrawElements with GPU-resident position and index buffers, where the shared center vertex is stored only ONCE.

Hints:

- A triangle fan's first vertex is the shared center — every triangle in the fan reuses it.
- Your index buffer should list the center index first, then the outer ring vertices in order, closing back to the first outer vertex.

Hard Exercises

Exercise Q15 — Continuous Bouncing glMapBuffer Animation [Hard]

Objective: Practice a self-sustaining glutTimerFunc + glMapBuffer animation loop with direction reversal.

Task (Question): Animate a quad's height so it bounces up and down continuously (reversing direction at top and bottom limits), using glutTimerFunc to repeatedly map, edit, and unmap its VBO.

Hints:

- Track the current direction (up or down) and current offset in global variables outside display().
- Reverse the direction when the offset passes a chosen minimum or maximum value, then continue animating.

Exercise Q16 — Three-Shape VBO Scene: Sun, Indexed Ground, Tree [Hard]

Objective: Practice a multi-shape scene mixing glDrawArrays and glDrawElements across independent VBOs.

Task (Question): Build a scene with at least 3 shapes (e.g., a sun, a ground rectangle, and a tree) where EVERY shape is sourced from a VBO, mixing glDrawArrays and glDrawElements as appropriate, and the ground uses an indexed element buffer.

Hints:

- A sun or star shape is a natural fit for glDrawElements with a shared center vertex.
- Keep each shape's buffer setup in its own init function so main() stays readable.

Exercise Q17 — Procedural 30-Dash Ring in One VBO [Hard]

Objective: Practice generating a larger batch of primitives procedurally and uploading them all into ONE buffer.

Task (Question): Generate 30 short dash segments arranged in a ring (using sinf/cosf in a loop) and upload them all into a SINGLE VBO, rendered with one GL_LINES glDrawArrays call.

Hints:

- Compute each dash's two endpoints from an angle that increases by (360 / 30) degrees per iteration.
- Allocate your GLfloat array BEFORE the loop, sized for 30 dashes x 2 points x however many floats per point you use.

Exercise Q18 — Interleaved + Indexed Hexagon [Hard]

Objective: Practice combining EVERY VBO technique from this module in a single shape.

Task (Question): Build an interleaved position+color VBO (like Example 12) for a hexagon rendered with glDrawElements AND a non-zero stride, using a separate GPU-resident index buffer.

Hints:

- Six vertices, no shared/duplicate positions needed here — the index buffer can simply be {0,1,2,3,4,5}.
- Double-check your stride math: with 6 floats per vertex, stride = 6 * sizeof(GLfloat), and the color pointer starts 3 floats after the vertex pointer.

Exercise Q19 — Keyboard-Triggered Color Change via glMapBuffer [Hard]

Objective: Practice using glMapBuffer in response to a keyboard event rather than a timer.

Task (Question): Draw a shape from a position VBO and a color VBO, and use glutKeyboardFunc so pressing the 'c' key maps the COLOR buffer, cycles every vertex to a new color, and unmaps it, updating the display immediately.

Hints:

- Call glutPostRedisplay() right after unmapping so the color change is visible without waiting for another event.
- Keep a global variable tracking which color scheme is currently active, and cycle it each time 'c' is pressed.

Exercise Q20 — Capstone: Animated Ship Over Indexed Ground [Hard]

Objective: Practice combining animation, indexing, multiple buffers, and cleanup into one complete program.

Task (Question): Build a capstone scene: an animated 'ship' triangle (position+color VBO, moved every second via glutTimerFunc + glMapBuffer), a static indexed ground quad (position VBO + index VBO), and an ESC-key handler that releases all four buffers before exit(0).

Hints:

- Keep the ship's animation state (e.g., current x offset) in a global variable updated inside the timer callback.
- Count your buffers carefully before writing the cleanup handler — this capstone uses FOUR separate GLuint IDs in total.

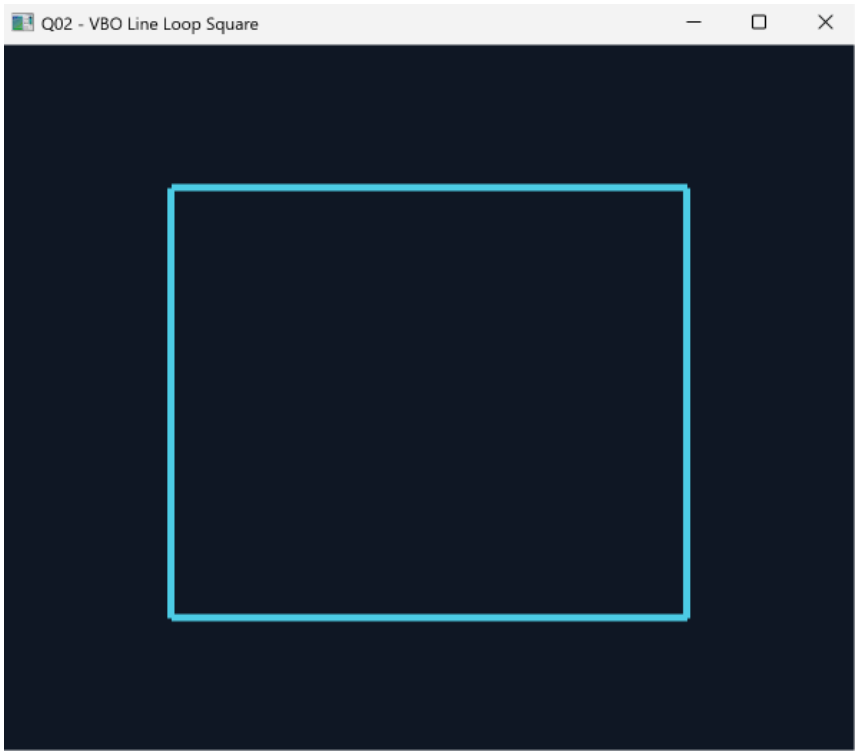
Part C — Outputs / Screen Shots (Ex01–Ex20)

Easy Exercises

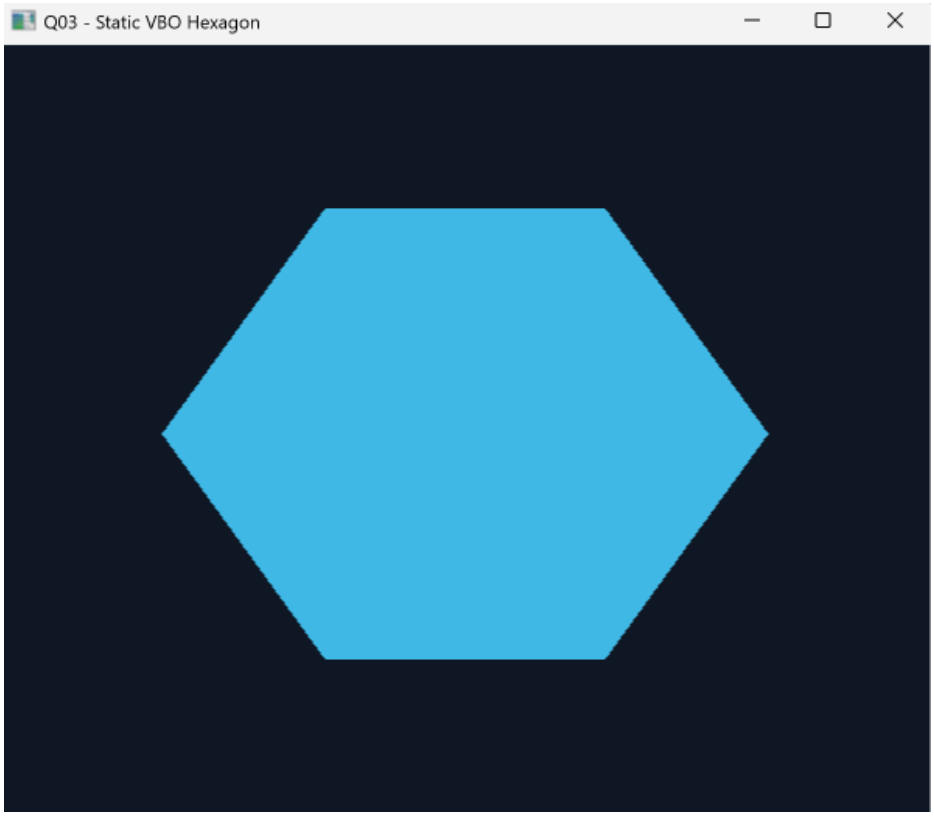
Exercise Q01 — VBO X-Pattern Points [Easy]



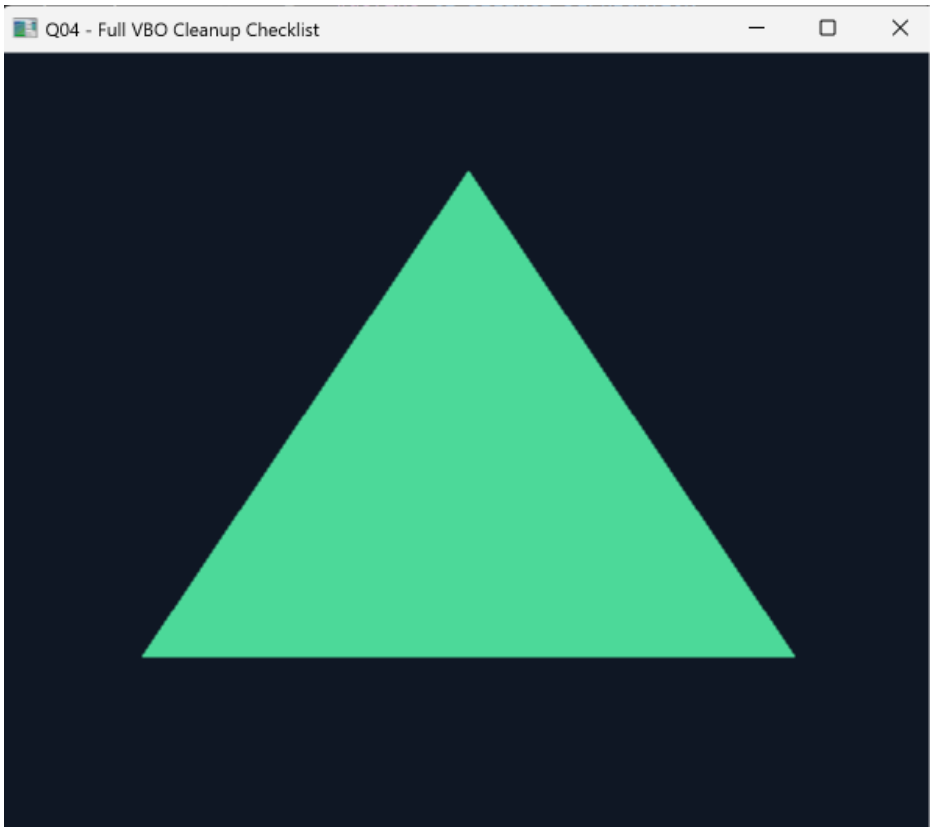
Exercise Q02 — VBO Line Loop Square [Easy]



Exercise Q03 — Static Hexagon with the Correct Usage Hint [Easy]



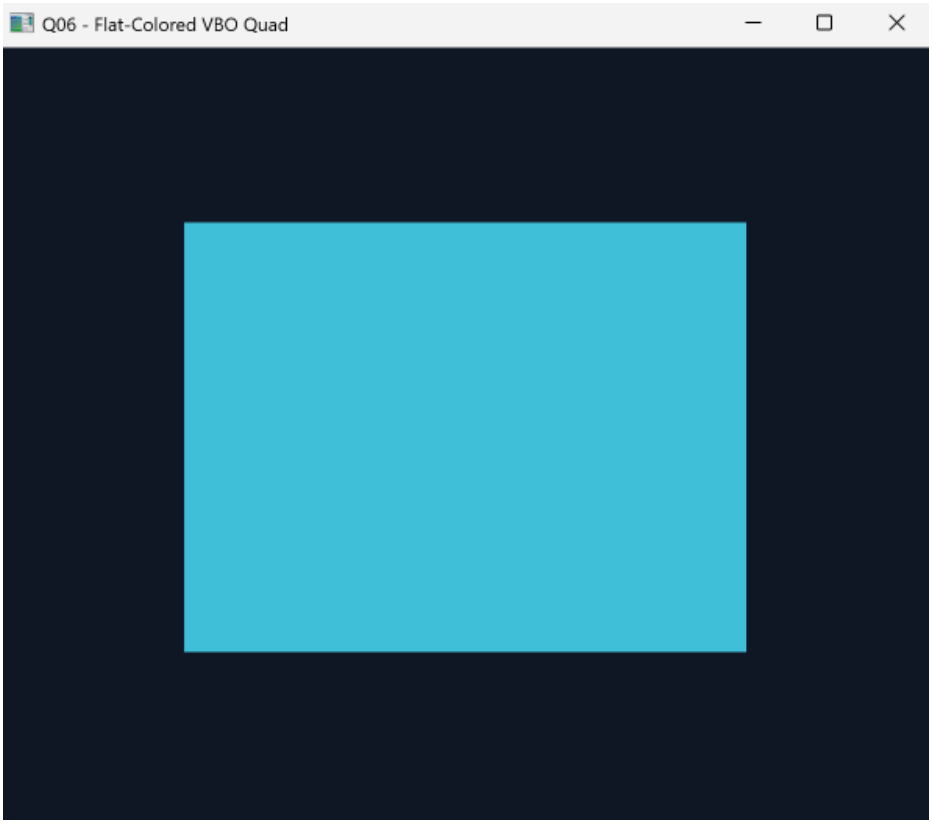
Exercise Q04 — Full Cleanup Checklist [Easy]



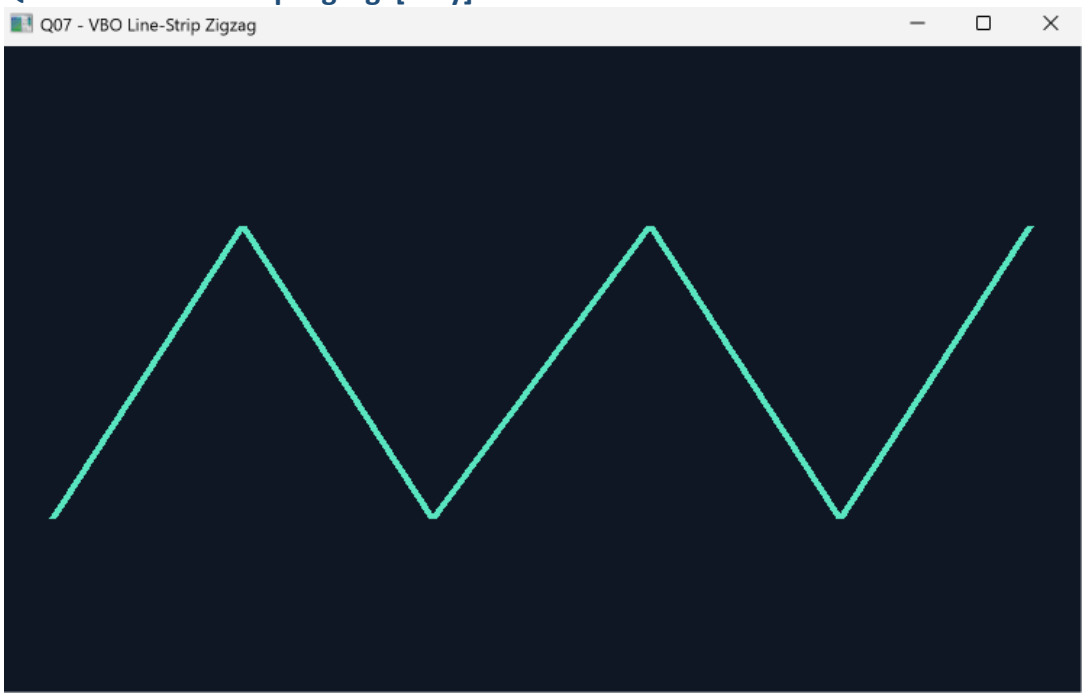
Exercise Q05 — Two Separate VBOs Side by Side [Easy]



Exercise Q06 — Flat-Colored VBO Quad [Easy]

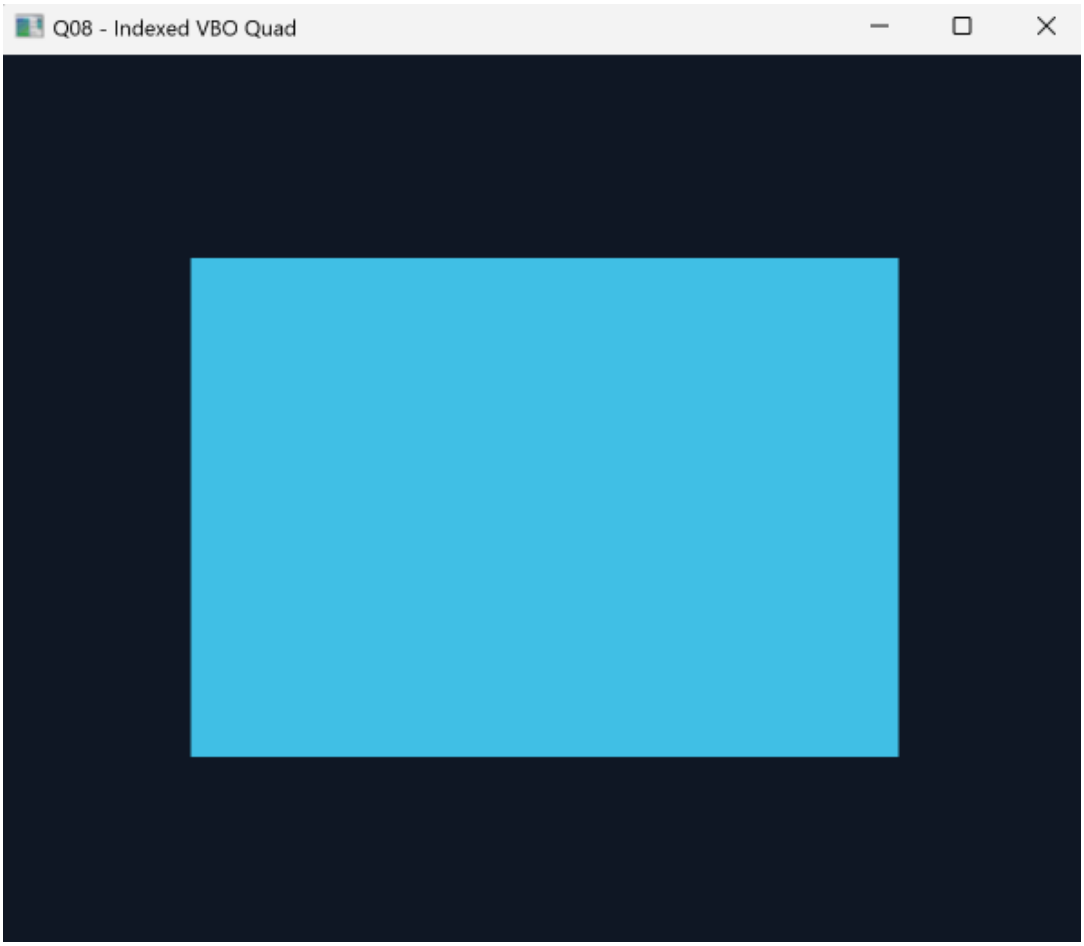


Exercise Q07 — VBO Line-Strip Zigzag [Easy]

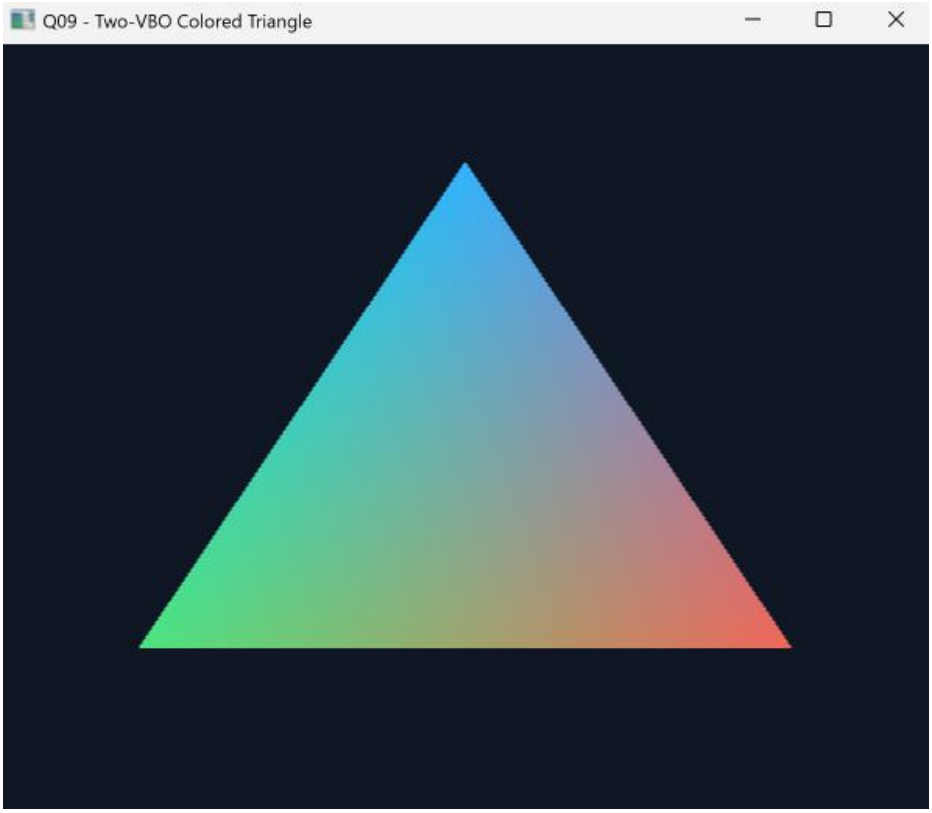


Medium Exercises

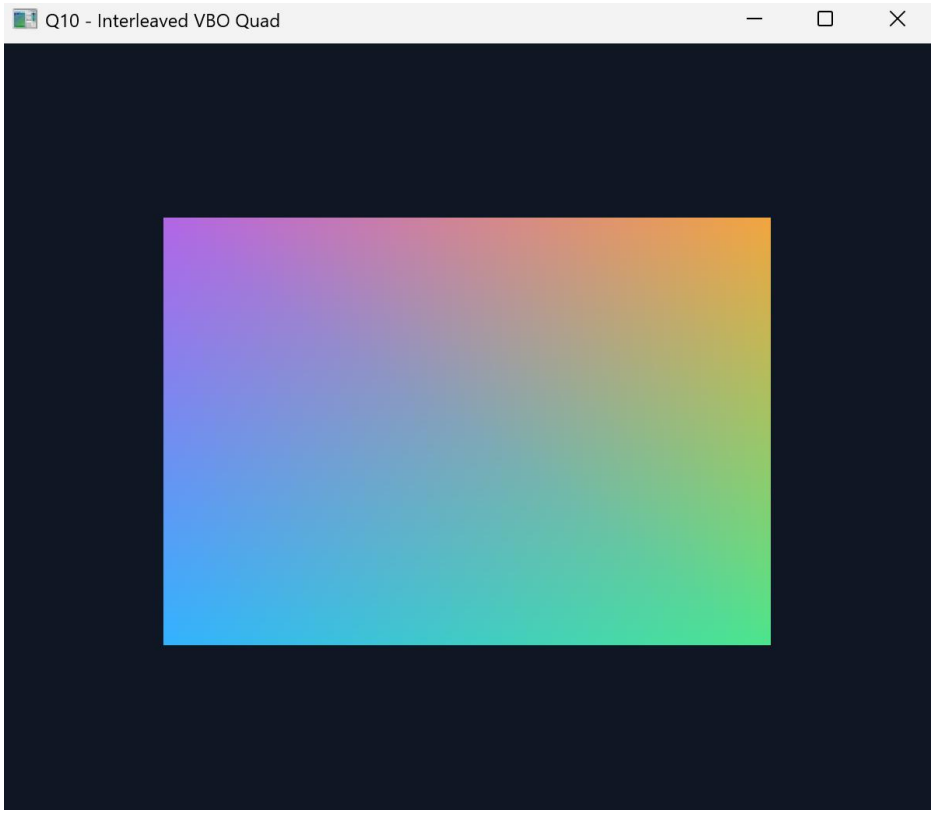
Exercise Q08 — Indexed Quad: Position VBO + Separate Index VBO [Medium]



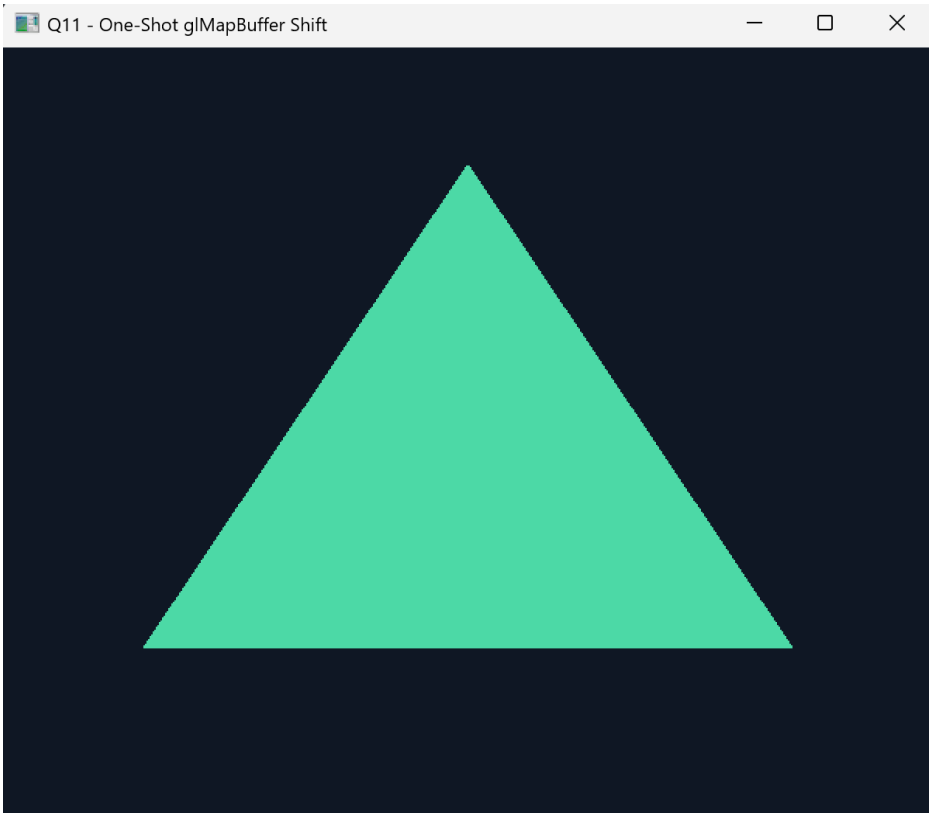
Exercise Q09 — Two-VBO Colored Triangle [Medium]



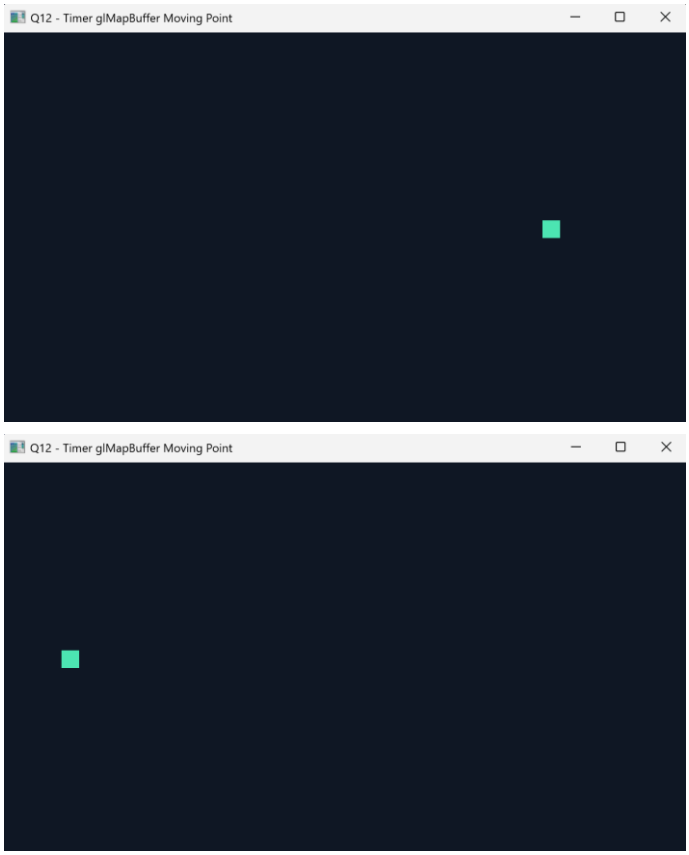
Exercise Q10 — Interleaved VBO Quad [Medium]



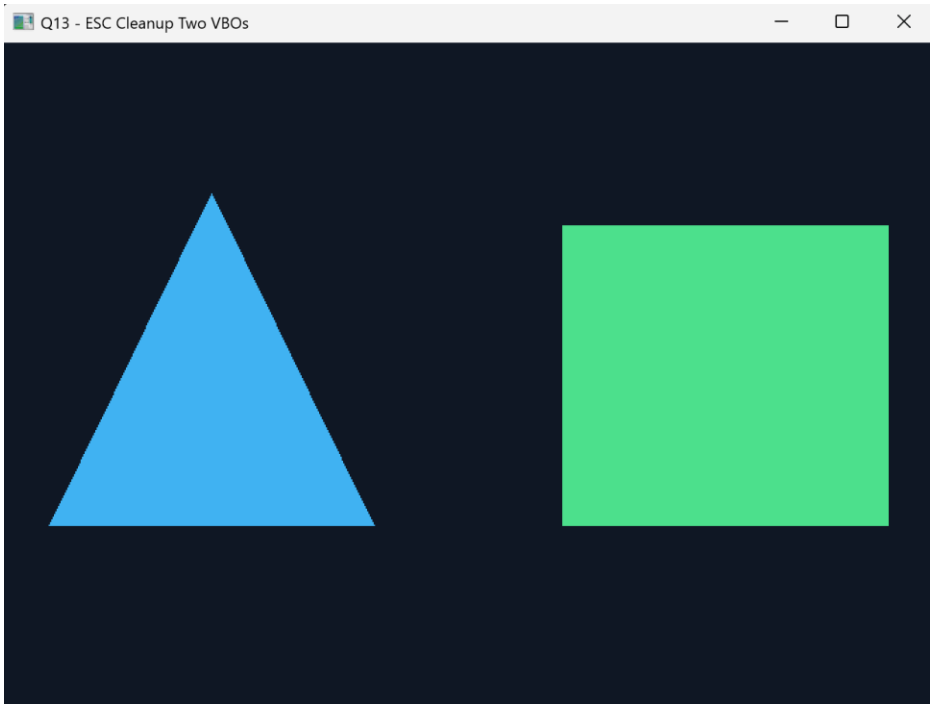
Exercise Q11 — One-Shot glMapBuffer Vertical Shift [Medium]



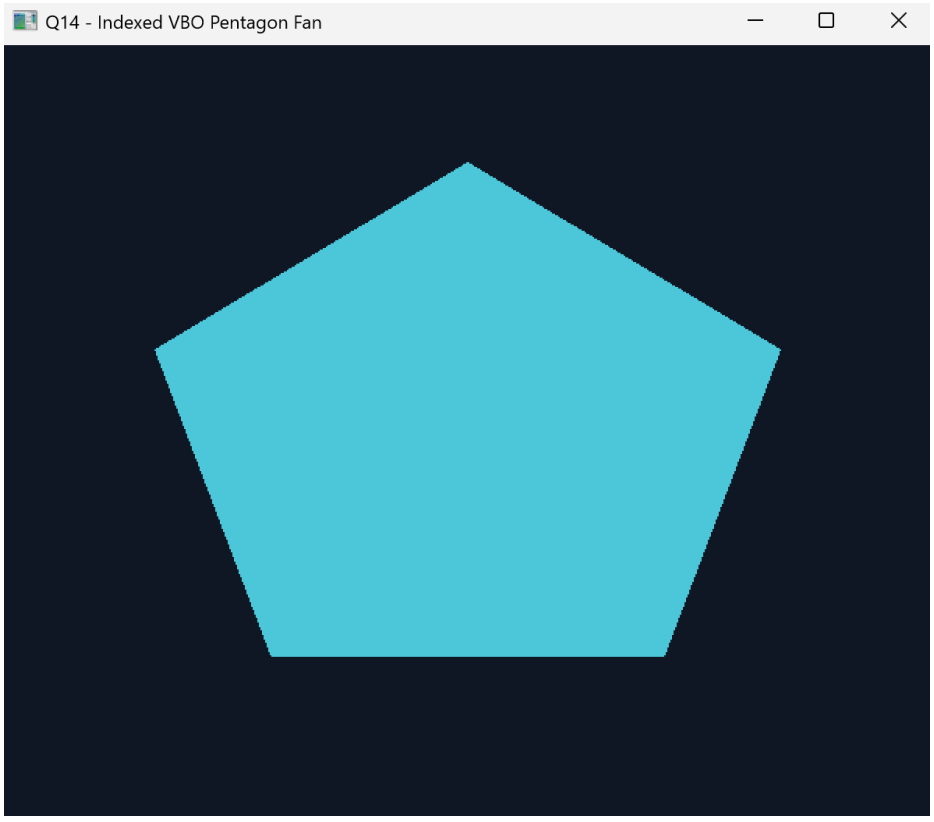
Exercise Q12 — Timer + glMapBuffer Moving Point [Medium]



Exercise Q13 — ESC Cleanup Releasing Two VBOs [Medium]

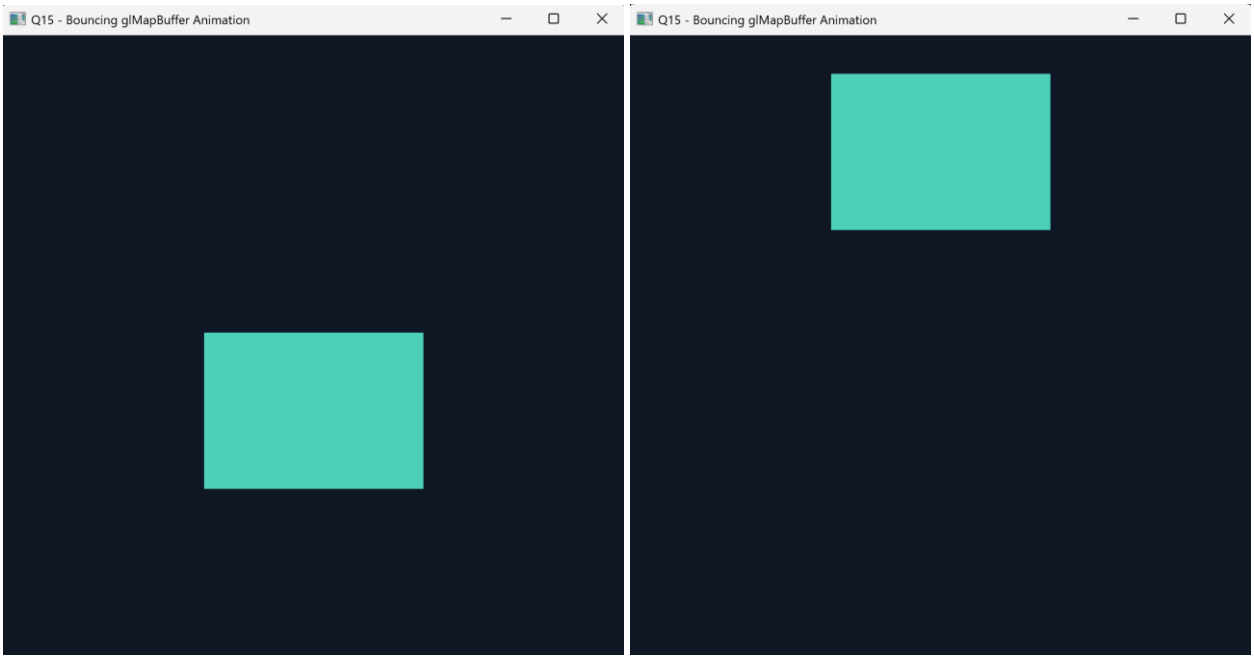


Exercise Q14 — Indexed GL_TRIANGLE_FAN Pentagon [Medium]

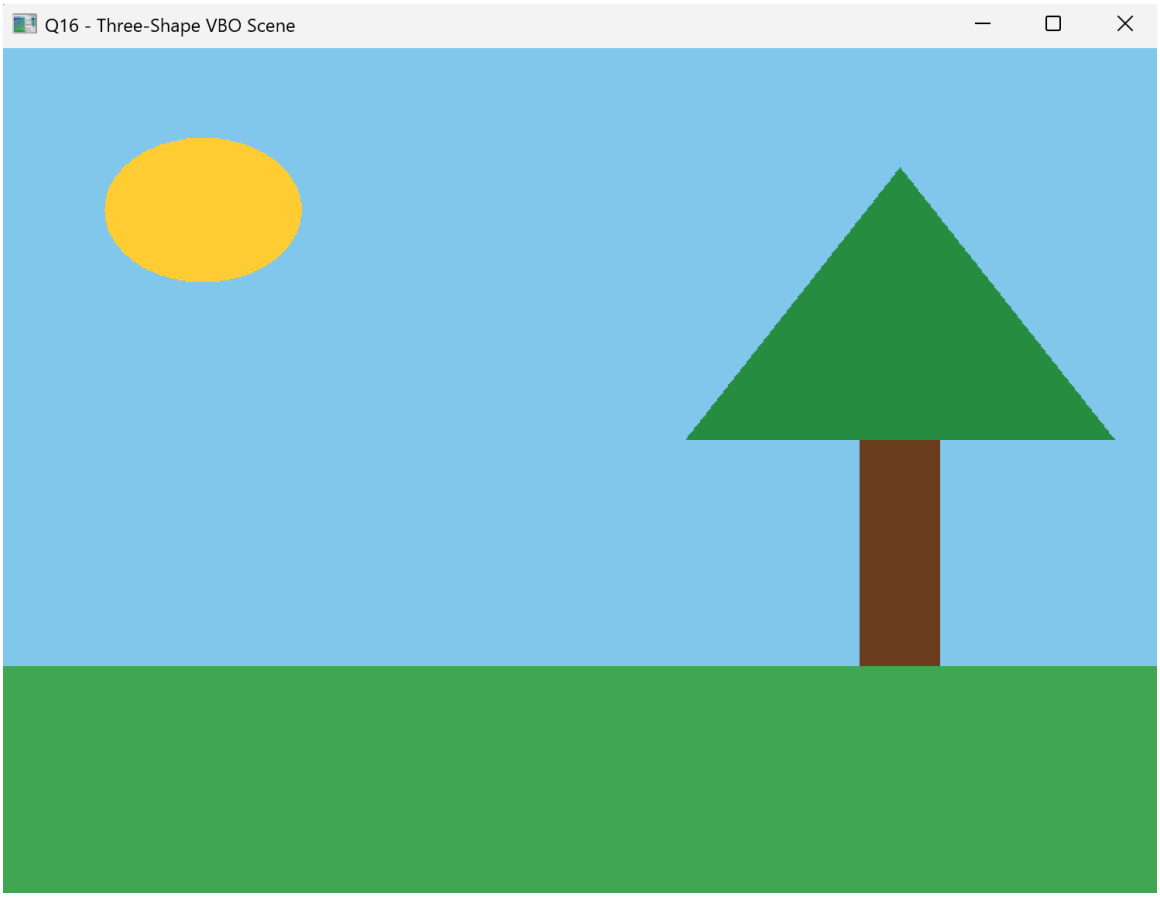


Hard Exercises

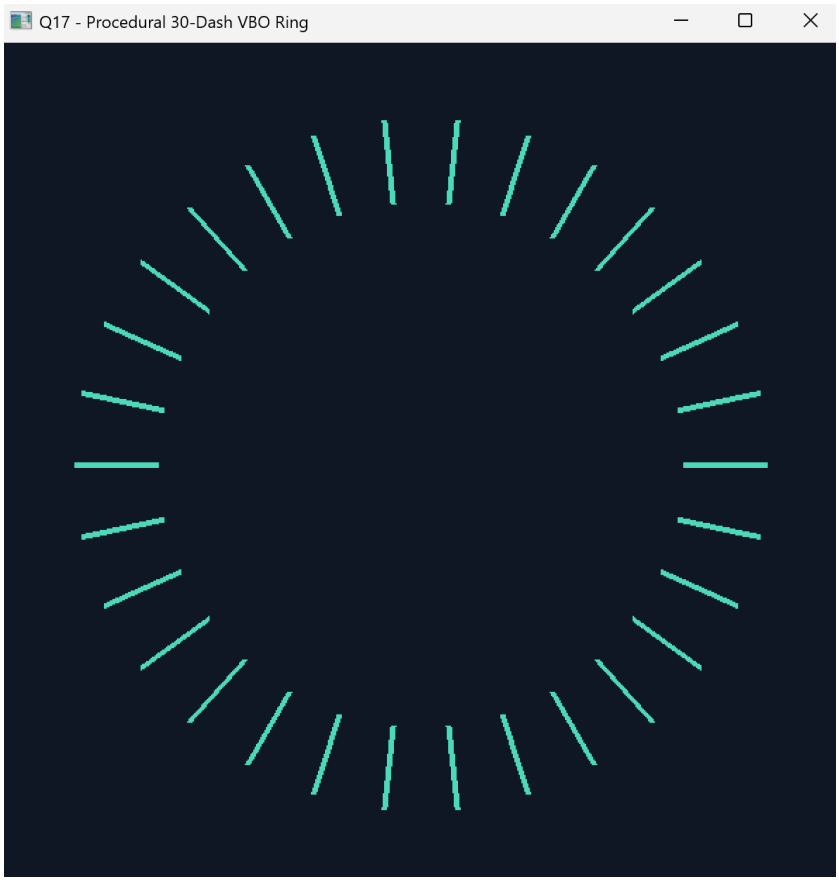
Exercise Q15 — Continuous Bouncing glMapBuffer Animation [Hard]



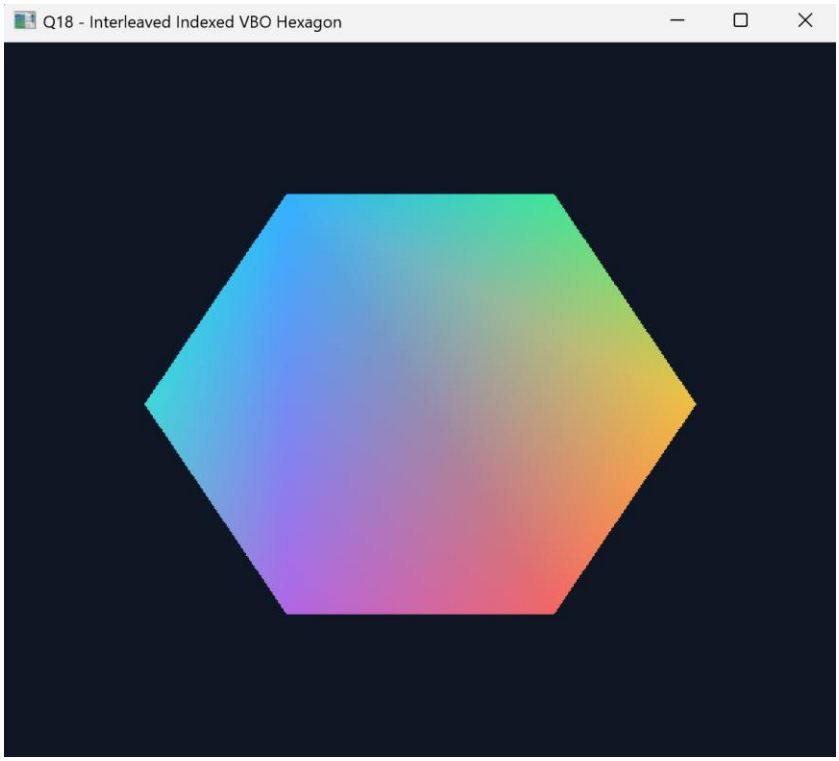
Exercise Q16 — Three-Shape VBO Scene: Sun, Indexed Ground, Tree [Hard]



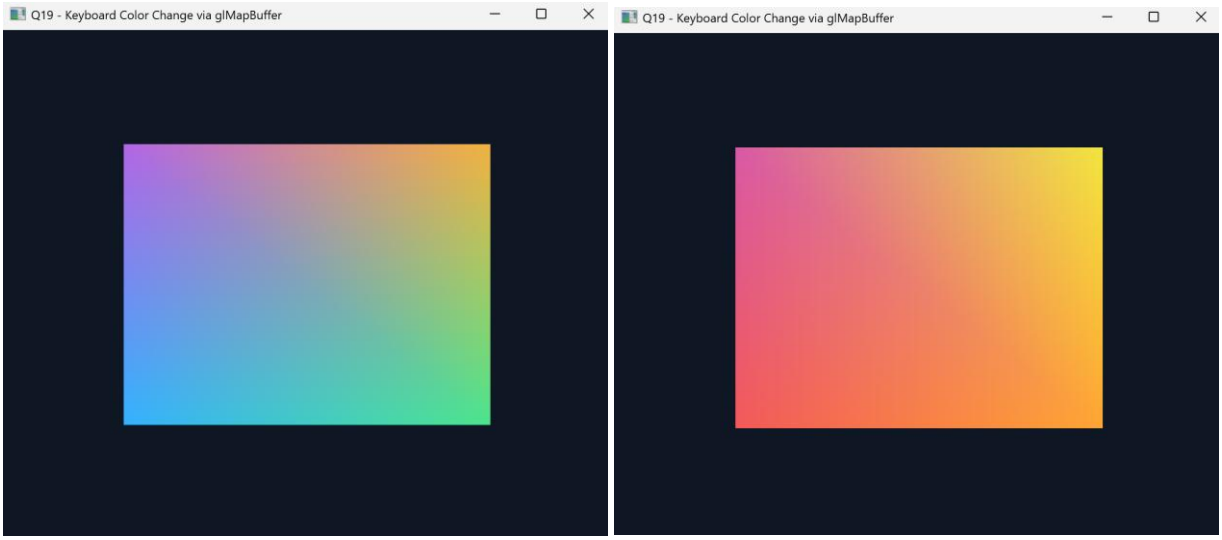
Exercise Q17 — Procedural 30-Dash Ring in One VBO [Hard]

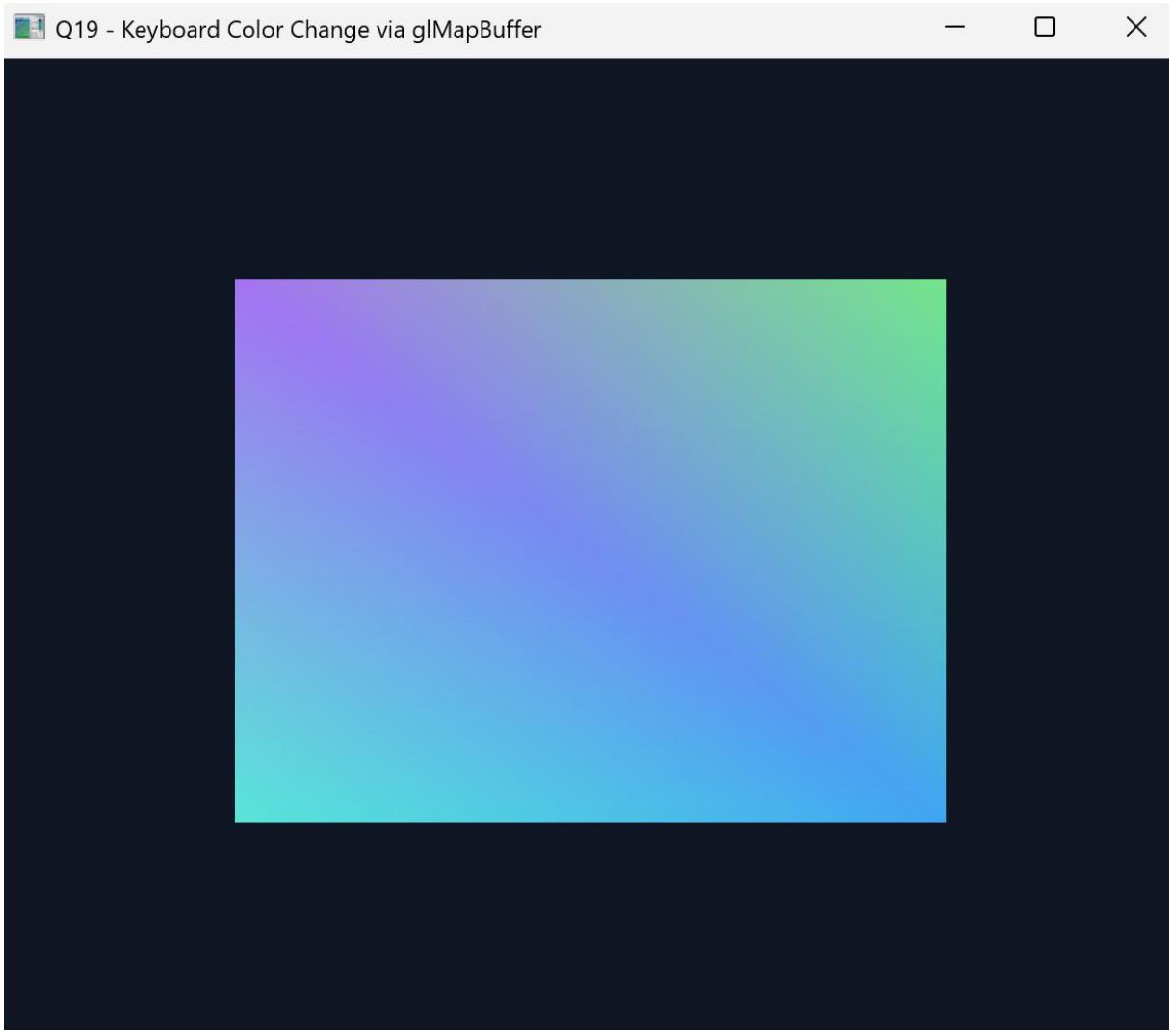


Exercise Q18 — Interleaved + Indexed Hexagon [Hard]



Exercise Q19 — Keyboard-Triggered Color Change via glMapBuffer [Hard]





Exercise Q20 — Capstone: Animated Ship Over Indexed Ground [Hard]

